Université Mohammed V de Rabat
École Nationale Supérieure d'Informatique et d'Analyse des Systèmes
Département Ingénierie des Systèmes Embarqués
Filière Smart Systems Engineering

Tutorial 1 : Prise en main de l'Arduino Portenta H7 et de la caméra avec MicroPython dans OpenMV IDE

Séries de tuto sur Portenta H7, lecture 1 « Introduction à l'Edge computing »

Pr.Hassan BERBIA/Pr.Aissam BERRAHOU

31 octobre 2026

Objectifs du Tutorial

Dans ce tutoriel, vous allez donner vie à la carte qui sera le cerveau de la station. L'objectif est de mettre en service l'unité de décision de la station EcoTri 4.0 : la carte Arduino Portenta H7 équipée du Vision Shield, programmée en MicroPython avec OpenMV IDE, et de relier ce que l'on observe aux notions du cours (mémoire, frame buffer, latence).
À la fin de ce tutoriel, vous serez capable de :

- Installer OpenMV IDE et mettre à jour le firmware OpenMV de la Portenta H7.
- Exécuter un premier script MicroPython (LED RGB) et utiliser le terminal série.
- Configurer la caméra : format de pixel, résolution, fenêtrage.
- Mesurer la cadence (FPS), le temps de capture et la mémoire disponible.
- Comprendre où sont stockées l'image (frame buffer) et les variables Python (tas).
- Enregistrer une image et lancer un programme autonome (`main.py`).

Durée indicative : 1 h 15. Ce tutoriel prépare le TP1 (mesures sur les trois cartes).

1 Introduction sur Portenta H7 et le Vision Shield

Le **Portenta H7** est une carte de développement haute performance conçue pour les applications industrielles, IoT et embarquées. Elle intègre un **double cœur Cortex-M7/M4**, du Wi-Fi, du **Bluetooth Low Energy (BLE)** via le module Murata 1DX, et supporte plusieurs environnements de développement, dont **MicroPython** grâce au firmware **OpenMV**. Associée au **Vision Shield**, elle devient une caméra intelligente : c'est l'**unité de décision** de la station EcoTri 4.0.

Caractéristiques principales :

- **Microcontrôleur** : STM32H747 double cœur (Cortex-M7 à 480 MHz + Cortex-M4 à 240 MHz), FPU double précision et instructions DSP.
- **Mémoire** : 2 Mo de Flash + 1 Mo de RAM internes ; 8 Mo de SDRAM et 16 Mo de Flash QSPI externes.
- **Connectivités** :
  - WiFi 802.11 b/g/n
  - Bluetooth 5.0 (BLE) grâce au module Murata 1DX
- **Interfaces** : USB-C, I²C, SPI, UART, CAN, etc.
- **Vision Shield** : capteur d'image monochrome 320×320 très basse consommation (Himax HM01B0 ou HM0360 selon la révision), deux microphones MEMS, lecteur microSD, Ethernet ou LoRa selon la version.

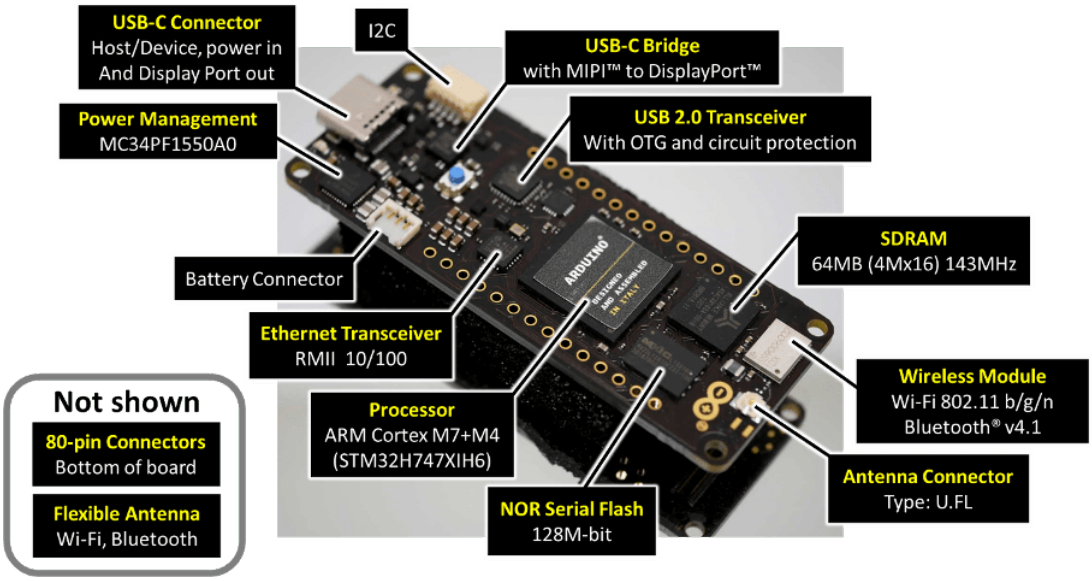


Figure 1: *
Portenta H7 : processeur STM32H747, SDRAM et Flash QSPI externes, module sans fil Murata, connecteurs USB-C et haute densité.

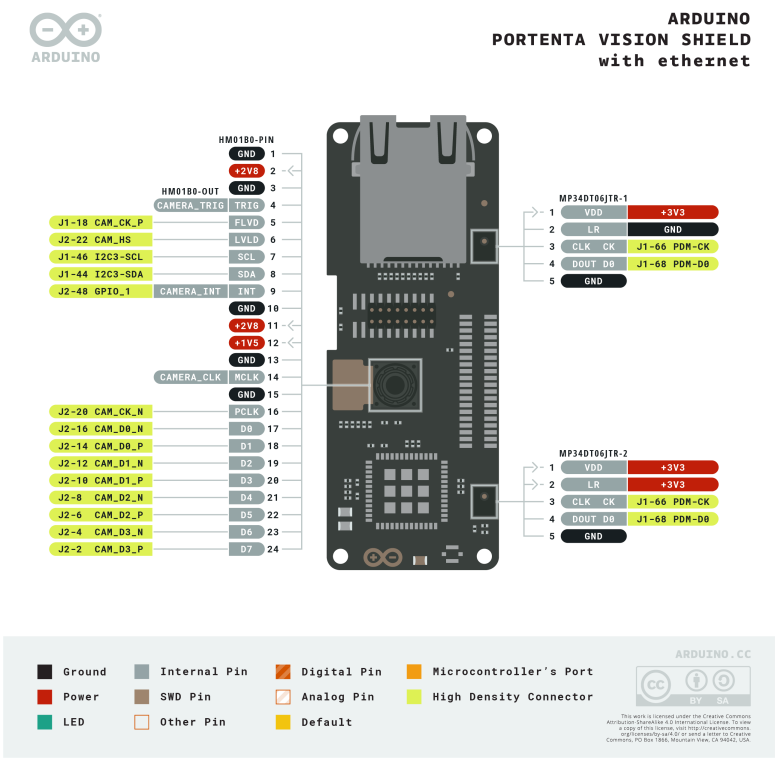
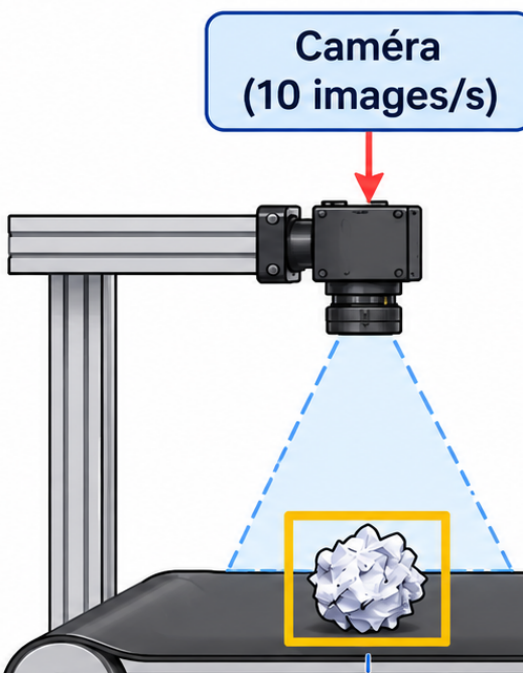


Figure 2: *
Vision Shield (version Ethernet) : caméra, micros PDM, microSD. Il se monte sur les deux connecteurs haute densité situés sous la Portenta.

Ce que nous allons traiter



Dans la station EcoTri 4.0, la Portenta et sa caméra observent le tapis. Ce tutoriel s'intéresse à ce seul maillon : capturer l'image de l'objet sous la caméra, et savoir combien de temps et de mémoire cela coûte.

Rappel du cours : où va quoi ?

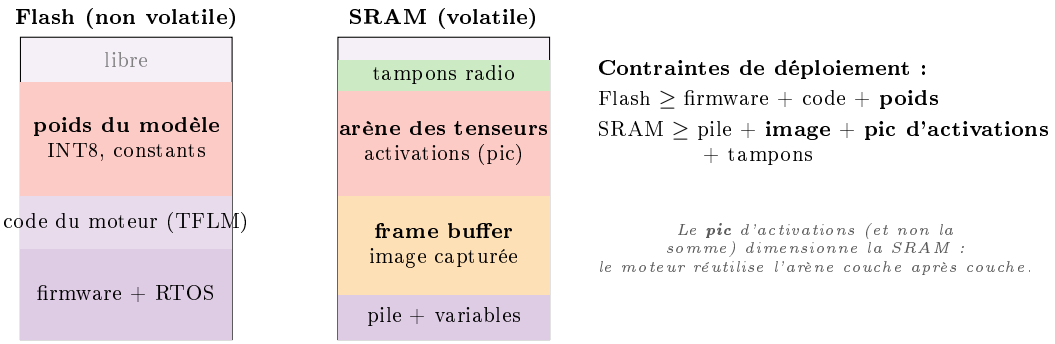


Figure 3: *

Le firmware OpenMV et vos scripts sont en Flash. À l'exécution, l'image capturée est placée dans le **frame buffer** et vos objets Python dans le **tas** (heap) : ce sont deux zones différentes de la RAM. Nous allons les observer.

2 Pré-requis

Matériel

- Carte **Arduino Portenta H7** et **Vision Shield**.
- Câble USB-C (données, pas seulement charge).
- Carte microSD (optionnelle, formatée en FAT32) pour l'étape 7.

Logiciel

1. **OpenMV IDE (version 4.0 ou plus)**.
   - Téléchargement : https://openmv.io/pages/download
2. **Firmware MicroPython (OpenMV) pour Portenta H7** :
   - Disponible via OpenMV IDE ou : https://github.com/openmv/openmv/releases
   - Documentation de l'API : https://docs.openmv.io

3 Étapes détaillées du tutoriel

Étape 1: Installer, connecter et mettre à jour le firmware

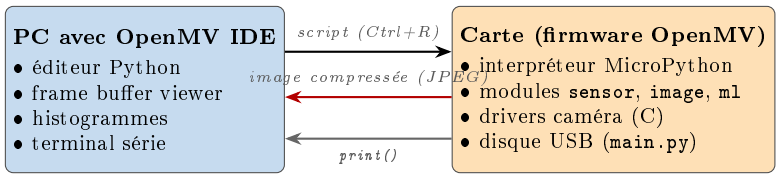
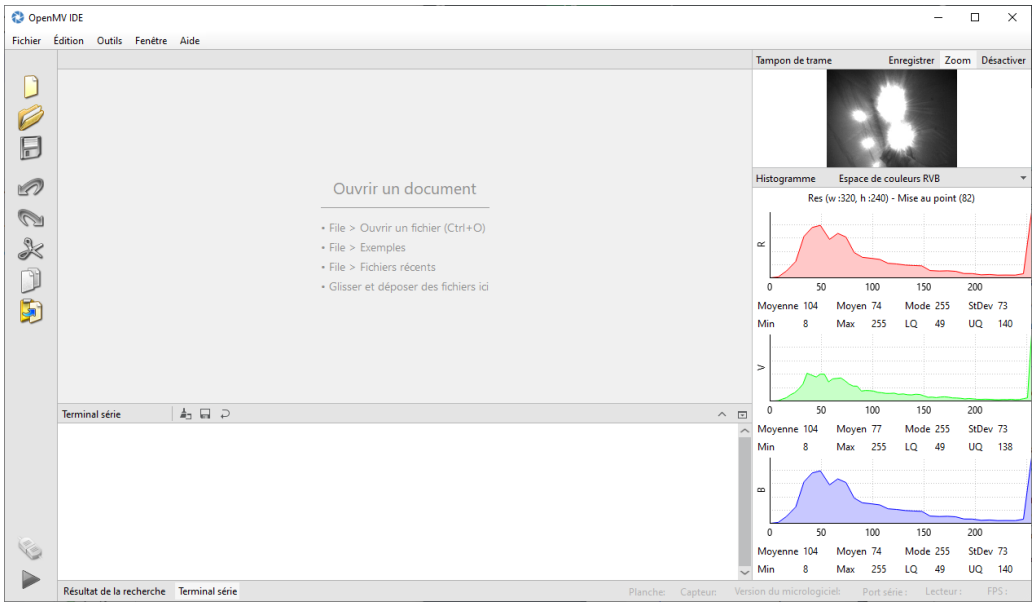


Figure 4: *
Principe d'OpenMV IDE : le script est envoyé à l'interpréteur MicroPython de la carte ; la carte renvoie l'image (compressée en JPEG) et les messages `print()` par le câble USB.

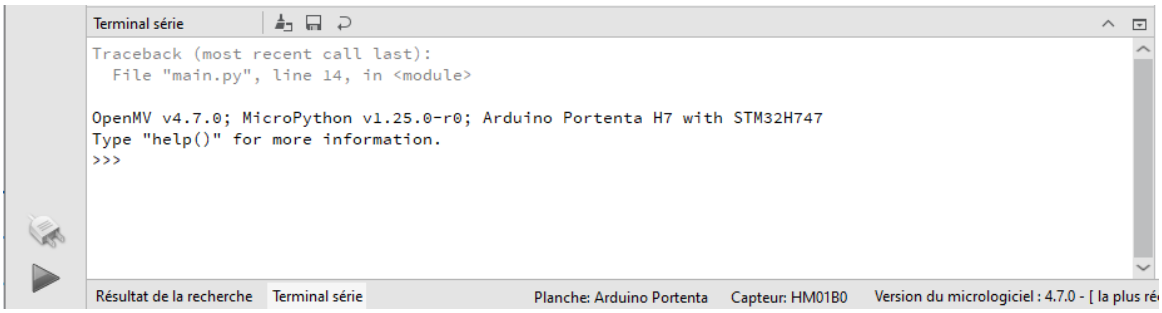
1. Montez le Vision Shield sur les connecteurs haute densité de la Portenta (bien aligner les deux connecteurs, appuyer franchement).
2. Branchez la Portenta H7 à votre ordinateur via USB-C.
3. Ouvrez OpenMV IDE.



4. Cliquez sur l'icône **Connect** (en bas à gauche).



5. Si la carte est reconnue, le terminal affiche la version de MicroPython/OpenMV et la carte détectée :



6. Si l'IDE propose une mise à jour du firmware, acceptez-la et laissez le processus se terminer (ne pas débrancher).

7. Si la carte n'est pas reconnue : mettez-la en mode bootloader en **appuyant deux fois rapidement** sur le bouton reset (la LED verte « respire »), puis utilisez **Tools** → **Run Bootloader (Load Firmware)** dans OpenMV IDE.

8. Une fois connecté, notez dans le terminal (**Tools** → **Open Terminal** ou panneau *Serial Terminal*) la version du firmware et le nom du capteur détecté.

Astuce

Le Vision Shield doit être monté **avant** la mise sous tension : le capteur est détecté au démarrage.

Étape 2: Premier script avec les LED RGB

Créez un nouveau fichier (**File** → **New File**), collez le code, puis lancez-le avec le bouton vert **Start** (Ctrl+R).

```
import time
from pyb import LED

red_led = LED(1)
green_led = LED(2)
blue_led = LED(3)

def led_off(): red_led.off(); green_led.off(); blue_led.off()
def led_red(): red_led.on(); green_led.off(); blue_led.off()
def led_green(): red_led.off(); green_led.on(); blue_led.off()
def led_blue(): red_led.off(); green_led.off(); blue_led.on()

compteur = 0
while True:
    led_blue();  time.sleep_ms(300)
    led_red();   time.sleep_ms(300)
    led_green(); time.sleep_ms(300)
    compteur += 1
    print("Cycle", compteur)
```

- `pyb.LED(n)` pilote les LED de la carte (1 : rouge, 2 : verte, 3 : bleue).
- `print()` s'affiche dans le terminal série de l'IDE.
- Le bouton rouge **Stop** interrompt le script (exception `KeyboardInterrupt`).

Étape 3: Identifier la carte et sa mémoire

Avant de travailler avec la caméra, interrogeons la carte sur ses ressources.

```
import sys, os, gc, machine, pyb

print("Plateforme   :", sys.platform, "| MicroPython", sys.version)
print("Implémentation:", os.uname().machine)
print("Fréquences   :", machine.freq())   # (SYSCLK, HCLK, PCLK1, PCLK2) en Hz

gc.collect()
libre = gc.mem_free()
occupe = gc.mem_alloc()
print("Tas Python   : %d o libres / %d o au total" % (libre, libre + occupe))

st = os.statvfs("/")                      # système de fichiers interne (flash)
print("Flash utilisateur : %d Ko libres" % (st[0] * st[3] // 1024))
```

Question

Relevez la fréquence du cœur et la taille du tas. Pourquoi le tas MicroPython est-il bien plus petit que les 1 Mo de SRAM + 8 Mo de SDRAM annoncés ? (Indice : le firmware réserve de la mémoire pour la pile, les pilotes, le frame buffer et le tampon de détection de l'IDE.)

Étape 4: Capturer des images

Ouvrez l'exemple **File** → **Examples** → **HelloWorld** ou tapez le code suivant :

```
import sensor, time

sensor.reset()                            # initialise le capteur
sensor.set_pixformat(sensor.GRAYSCALE)    # le capteur du Vision Shield est monochrome
sensor.set_framesize(sensor.QVGA)         # 320x240
sensor.skip_frames(time=2000)             # laisse l'exposition automatique converger
clock = time.clock()

print("Capteur :", hex(sensor.get_id()),
      "| taille :", sensor.width(), "x", sensor.height())

while True:
    clock.tick()                          # démarre la mesure de la boucle
    img = sensor.snapshot()               # capture -> frame buffer
    print("FPS : %.1f" % clock.fps())
```

- L'image apparaît dans le **Frame Buffer** en haut à droite de l'IDE ; l'histogramme est affiché en dessous.
- `sensor.snapshot()` renvoie un objet `image` qui pointe vers le frame buffer : aucune copie n'est faite.
- `clock.fps()` donne la cadence moyenne de la boucle.

Astuce

Le capteur du Vision Shield est **monochrome** : `sensor.RGB565` n'apporte pas de couleur. Si `set_framesize` refuse une taille, consultez `docs.openmv.io` pour les tailles supportées par votre capteur.

Étape 5: Mesurer FPS, temps de capture et mémoire

Nous voulons séparer le **temps de capture** du **temps d'affichage** dans l'IDE, et vérifier le coût mémoire $M = W \times H \times b$ vu en cours.

```
import sensor, time, gc

TAILLES = [("QQVGA", sensor.QQVGA), ("QVGA", sensor.QVGA), ("B320X320", sensor.
    B320X320)]
N = 50                                  # images par mesure

sensor.reset()
sensor.set_pixformat(sensor.GRAYSCALE)

for nom, taille in TAILLES:
    try:
        sensor.set_framesize(taille)
    except Exception as e:
        print(nom, ": non supporté", e); continue
    sensor.skip_frames(time=1000)
    t0 = time.ticks_us()
    for i in range(N):
        img = sensor.snapshot()
    dt = time.ticks_diff(time.ticks_us(), t0) / N    # µs par image
    gc.collect()
    print("%-9s %3dx%3d | %6d o/image | capture %6.1f ms | %5.1f FPS | tas libre %d o
    "
          % (nom, img.width(), img.height(), img.size(), dt / 1000, 1e6 / dt, gc.
    mem_free()))
```

1. Exécutez le script avec le **Frame Buffer activé**, puis désactivez-le (bouton **Disable** au-dessus de l'image) et recommencez.
2. Vérifiez que `img.size()` vaut bien $W \times H \times 1$ octet en niveaux de gris.
3. Remarquez que le tas libre ne dépend presque pas de la résolution : l'image n'est pas dans le tas.

Résolution	Octets / image	Capture (ms)	FPS (IDE actif)	FPS (IDE désactivé)
QQVGA 160×120				
QVGA 320×240				
320×320				

Table 1: *
Tableau à compléter (il sera réutilisé dans le TP1).

Question

Pourquoi les FPS augmentent-ils quand l'affichage est désactivé ? Reliez votre réponse au cours : quelle ressource est consommée par l'envoi de l'image vers le PC ?

Étape 6: Réduire le coût : fenêtrage et sous-échantillonnage

Un modèle TinyML travaille souvent en 96×96. On peut soit **fenêtrer** (ne lire qu'une région du capteur), soit **réduire** l'image après capture.

```
import sensor, time

sensor.reset()
sensor.set_pixformat(sensor.GRAYSCALE)
sensor.set_framesize(sensor.QVGA)
sensor.set_windowing((112, 72, 96, 96))  # (x, y, w, h) : région centrale 96x96
sensor.skip_frames(time=1000)
clock = time.clock()

while True:
    clock.tick()
    img = sensor.snapshot()
    img.draw_rectangle(0, 0, img.width(), img.height(), color=255)
    print("%dx%d  %d o  %.1f FPS" % (img.width(), img.height(), img.size(), clock.fps
    ()))
```

- `set_windowing` réduit la quantité de données transférées et traitées : le gain de FPS dépend du capteur et du pilote.
- Alternative : `img.scale(x_size=96)` (ou `img.resize` selon la version du firmware) réduit l'image après capture.

Étape 7: Enregistrer une image et lancer un programme autonome

```
import sensor, time
from pyb import LED

sensor.reset()
sensor.set_pixformat(sensor.GRAYSCALE)
sensor.set_framesize(sensor.QVGA)
sensor.skip_frames(time=2000)

LED(3).on()                                  # bleu : capture en cours
img = sensor.snapshot()
img.save("/capture_%d.jpg" % time.ticks_ms(), quality=90)  # flash interne ou microSD
LED(3).off(); LED(2).on()                    # vert : terminé
print("Image enregistrée")
```

1. Après exécution, la carte apparaît comme un **disque USB** sur votre ordinateur : retrouvez-y le fichier `.jpg`. Avec une microSD insérée, c'est la carte SD qui est montée.
2. Pour qu'un script démarre **sans PC**, utilisez **Tools → Save open script to OpenMV Cam (as main.py)**, puis réinitialisez la carte.
3. Comparez la taille du fichier JPEG à la taille brute (76 800 octets) : quel est le taux de compression ?

Astuce

Éjectez proprement le disque USB avant de débrancher la carte, sinon le système de fichiers peut être corrompu.

Étape 8: Relier les mesures au budget de la station

1. À partir du FPS mesuré en QVGA, calculez le temps de capture T_{capt} et sa part dans l'échéance $T_{\max} = 600$ ms de la station.
2. Calculez le débit $D = W \times H \times b \times f \times 8$ qu'il faudrait pour envoyer ces images brutes au cloud, puis en JPEG (taille mesurée à l'étape 7).
3. Concluez : la Portenta peut-elle capturer assez vite pour la station ? Reste-t-il du temps pour l'inférence ?

4 Pour aller plus loin

- Exemples OpenMV : **File** → **Examples** → **Snapshot** et **Image Processing**.
- Documentation OpenMV : docs.openmv.io (modules `sensor`, `image`, `time`).
- Arduino, *Getting Started with OpenMV and MicroPython on Portenta H7* : docs.arduino.cc.
- Harvard CS249r / *Machine Learning Systems* (V. Janapa Reddi), chapitre « Embedded AI » : mlsysbook.ai.